Docker compose for multi-container SpringBoot Application

1. Problem with Individual Container

- Previously, we created containers using separate commands
- This works when we have only few containers
- But as the application grows, we may have :
  - 5 containers
  - 10 containers
  - 20 containers
  - Managing all those **Docker Commands individually becomes cumbersome and difficult to maintain.**
- **Solution : Docker Compose**

2. What is Docker Compose?

- **Docker compose is a tool for defining and managing muti-container Docker applications**
- Instead of maintaining many **Docker Commands,** we define everything in **one YAML file**
- We then **run** the **Compose file** and **Docker** takes care of **creating** and **starting** the required services.

3. What can we define in Docker Compose?

- A single YAML file can define
  1. Services
  2. Networks
  3. Volumes
  4. Docker images
  5. Ports
  6. Environment variables
  7. Container configuration
  8. Restart settings
- This makes the entire multi-container setup easier to maintain

4. Docker Compose YAML file

The configuration is stored in a YAML file, commonly:

```
docker-compose.yml
```

or:

```
docker-compose.yaml
```

In the lecture, the file is created as:

```
docker-compose.yaml
```

inside the **root directory of the project**. Pasted markdown

5. Services

- Services defines the different services that make up your application
- In Docker Compose terminology, each service represents a **container/service configuration.**
- In this example there are two services
  - PostgreSQL
  - PgAdmin

6. The Big Advantage

Without Compose:

```
Terminal
   ↓
docker network create ...
   ↓
docker run postgres ...
   ↓
docker run pgadmin ...
   ↓
Configure everything manually
```

With Compose:

```
docker-compose.yaml
        ↓
docker compose up
        ↓
All services + network + volumes
        ↓
Running application
```

This is much easier to maintain as the number of services grows.

7. IntelliJ Support

- Compose file is directly created in IntelliJ
- IntelliJ can recognize the Docker Compose YAML file
- With Docker support/plugin enabled, IntelliJ provides
  - Syntax Highlighting
  - Run options
  - Docker compose integration
- Installing or enabling the Docker Plugin is necessary if support isn’t available.

8. Docker compose up

This is the main command.

```bash
docker compose up
```

It starts the services defined under:

```yaml
services:
```

It also creates/starts the required:

- Network
- Volumes
- Containers

The lecture demonstrates that Docker pulls the required images if they aren't already available locally.

9. Start Only One Service

You don't always have to start everything.

PostgreSQL only

```bash
docker compose up postgres
```

pgAdmin only

```bash
docker compose up pgadmin
```

This allows you to selectively start individual services.

10. Detached Mode

Just like `docker run`, Compose can run in detached mode:

```bash
docker compose up -d
```

Without `-d`

```bash
docker compose up
```

- Logs appear in the terminal.
- Terminal remains attached to the Compose process.

With `-d`

```bash
docker compose up -d
```

- Services run in the background.
- Terminal remains available.

The lecture demonstrates this after stopping the foreground Compose process.

11. Check Running Containers

After:

```bash
docker compose up -d
```

you can run:

```bash
docker ps
```

This shows the running containers.

For example:

```
PostgreSQL container
pgAdmin container
```

The lecture demonstrates using `docker ps` to verify the containers are running.

12. View Logs

You can use:

Bash

```
docker logs <container-id>
```

to see the logs of an individual container.

For example:

Bash

```
docker logs <postgres-container-id>
```

or:

Bash

```
docker logs <pgadmin-container-id>
```

The lecture demonstrates this approach for both containers.

13. Docker Compose Down

When you want to stop the Compose application:

Bash

```
docker compose down
```

This stops/removes the Compose-managed containers and removes the Compose-created network.

The lecture demonstrates that after `docker compose down`, the containers are gone while the images remain.

Pasted markdown

Remember

```
docker compose up
       ↓
Start application

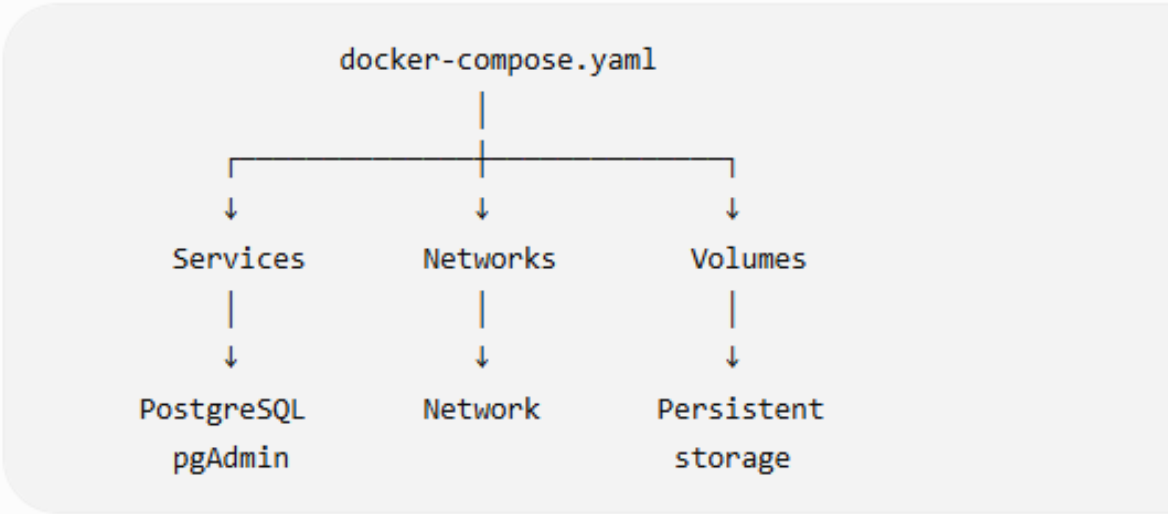
docker compose down
       ↓
Stop/remove Compose containers
```

14. Most Important Commands

Command	Purpose
`docker compose up`	Start all services
`docker compose up -d`	Start all services in background
`docker compose up postgres`	Start only PostgreSQL
`docker compose up pgadmin`	Start only pgAdmin
`docker compose down`	Stop/remove Compose application
`docker ps`	See running containers
`docker logs <container>`	View container logs

15. Most Important Concept

Think of **Docker Compose as a manager for multiple containers.**



Interview answer

> "Docker Compose is a tool used to define and manage multi-container Docker applications. We define services, networks, volumes, environment variables, ports, and other configurations in a YAML file, and then use commands such as `docker compose up` and `docker compose down` to manage the complete application."

🧠 One-line memory

> Docker `run` → manage containers individually.
> Docker Compose → manage multiple services from one YAML file.

Sources